

Experiment 1

Cryptography in Blockchain, Merkle Root Tree Hash

Aim

To understand cryptographic hash functions, Proof-of-Work, and Merkle Tree construction as used in Blockchain, by implementing SHA-256 hashing, a nonce-based target hash, a Proof-of-Work puzzle solver, and a Merkle Tree over a sample set of transactions.

Theory

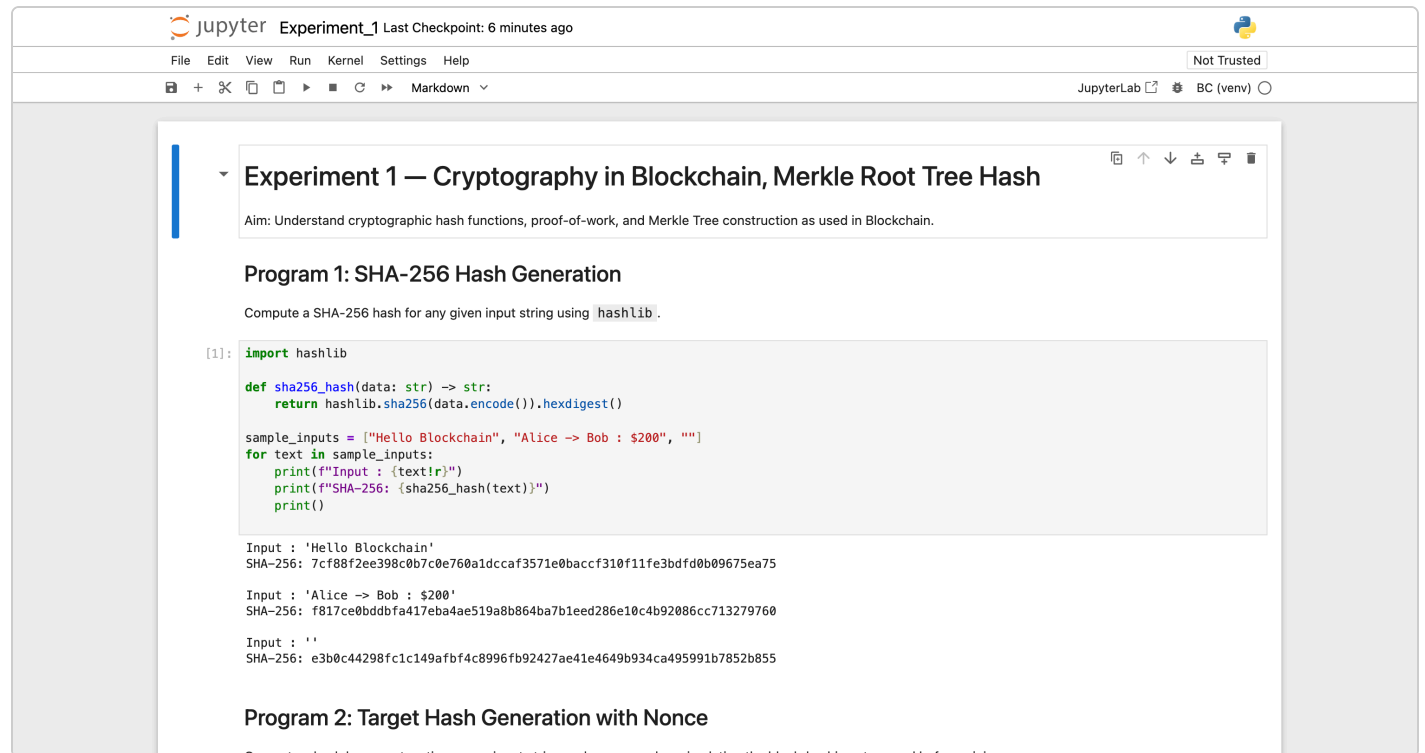
A cryptographic hash function such as SHA-256 maps an input of any length to a fixed-size (256-bit) digest that is deterministic, fast to compute in the forward direction, and infeasible to invert or find collisions for. Blockchain systems rely on this to fingerprint blocks and transactions: even a one-character change in the input produces a completely different, unpredictable hash (the avalanche effect).

Proof-of-Work is the mechanism Bitcoin-style blockchains use to make adding a new block computationally expensive. A miner repeatedly hashes a combination of the block's data and a changing **nonce** until the resulting hash satisfies a difficulty target — conventionally, a required number of leading zero bits/hex digits. The nonce that first satisfies this target is called the **Golden Nonce**. Because SHA-256 output is effectively random with respect to its input, the only way to find a qualifying nonce is brute-force search, and the expected number of attempts grows exponentially with the difficulty (number of required leading zeros).

A **Merkle Tree** is a binary tree of hashes used to efficiently and verifiably summarize a set of transactions. Each transaction is hashed individually (the leaves); pairs of hashes are then concatenated and re-hashed to form the parent level, repeating until a single hash — the **Merkle Root** — remains. When a level has an odd number of nodes, the last node is duplicated so it can still be paired. The Merkle Root is stored in the block header: it lets a node verify that a specific transaction belongs to a block using only a small number of intermediate hashes (a Merkle proof), without downloading every transaction in the block.

Tools used: Python 3.12, the standard `hashlib` library, run in a local Jupyter Notebook (`Experiment_1.ipynb`, in this folder).

Output



The screenshot shows a Jupyter Notebook titled "Experiment_1" with a last checkpoint 6 minutes ago. The notebook has a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main content area displays the following:

Experiment 1 — Cryptography in Blockchain, Merkle Root Tree Hash

Aim: Understand cryptographic hash functions, proof-of-work, and Merkle Tree construction as used in Blockchain.

Program 1: SHA-256 Hash Generation

Compute a SHA-256 hash for any given input string using `hashlib`.

```
[1]: import hashlib

def sha256_hash(data: str) -> str:
    return hashlib.sha256(data.encode()).hexdigest()

sample_inputs = ["Hello Blockchain", "Alice -> Bob : $200", ""]
for text in sample_inputs:
    print(f"Input : {text!r}")
    print(f"SHA-256: {sha256_hash(text)}")
    print()
```

Input : 'Hello Blockchain'
SHA-256: 7cf88f2ee398c0b7c0e760a1dcca3571e0baccf310f11fe3bdfd0b09675ea75

Input : 'Alice -> Bob : \$200'
SHA-256: f817ce0bdddafa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

Input : ''
SHA-256: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

Program 2: Target Hash Generation with Nonce

Generate a hash by concatenating a user input string and a nonce value, simulating the block hashing step used before mining.

Program 1: SHA-256 digests computed for three sample inputs, including the empty string, each producing a distinct 64-character hex digest.

jupyter

Experiment_1 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

+

+

+

+

+

+

Markdown

JupyterLab

BC (venv)

Program 2: Target Hash Generation with Nonce

Generate a hash by concatenating a user input string and a nonce value, simulating the block-hashing step used before mining.

```
[2]: def target_hash(data: str, nonce: int) -> str:
    combined = f"{data}{nonce}"
    return hashlib.sha256(combined.encode()).hexdigest()

data = "Block-1-Transactions"
for nonce in range(5):
    print(f"nonce={nonce} -> hash={target_hash(data, nonce)}")

nonce=0 -> hash=d57ae0b8ee9ac5e074b35fcfd1a338a1b5f5058bc1b6678a0231f550e136ff8d
nonce=1 -> hash=d779a69e31ba5797b8c2816234a1c87dcbbd5842bb4e3d5b5128af0cad9186ff
nonce=2 -> hash=b823c7e48d386dd863754d977d4ff7e20f14f87785d6d8bd7513aea431b4ef22
nonce=3 -> hash=0dbbd7ab0135020e0a3863d8985e7721f6f508ba4b1d8080505b4fd42d1fc422
nonce=4 -> hash=a7c1071a089a34e677f6ff64a6374324a64d7e3eec5fa4dd19b49bd685fd0ad3
```

Program 3: Proof-of-Work Puzzle Solving

Find the nonce that, combined with a given input string, produces a hash with a specified number of leading zeros — the 'Golden Nonce'.

```
[3]: def proof_of_work(data: str, difficulty: int = 4):
    prefix = "0" * difficulty
    nonce = 0
    while True:
        candidate = target_hash(data, nonce)
        if candidate.startswith(prefix):
            return nonce, candidate
        nonce += 1

data = "Block-1-Transactions"
difficulty = 4
golden_nonce, final_hash = proof_of_work(data, difficulty)
print(f"Data : {data}")
print(f"Difficulty : {difficulty} leading zeros")
```

Program 2: hash of `data + nonce` for nonce 0–4, showing how the hash changes unpredictably with the nonce. Program 3 (Proof-of-Work) begins below it.

jupyter

Experiment_1 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

+

+

+

+

+

+

Markdown

JupyterLab

BC (venv)

```
golden_nonce, final_hash = proof_of_work(data, difficulty)
print(f"Data : {data}")
print(f"Difficulty : {difficulty} leading zeros")
print(f"Golden Nonce : {golden_nonce}")
print(f"Resulting Hash : {final_hash}")

Data : Block-1-Transactions
Difficulty : 4 leading zeros
Golden Nonce : 53370
Resulting Hash : 0000b618a680c000b99996127e037fed0e6b86df5d67ca5003ab770c32f817e
```

Program 4: Merkle Tree Construction

Build a Merkle Tree from the sample transaction list below. Each transaction is hashed individually, then pairs of hashes are recursively combined and re-hashed (duplicating the last node when a level has an odd number of nodes) until a single Merkle Root remains. All intermediate hashes are printed.

```
[4]: transactions = [
    "Alice -> Bob : $200", # T1
    "Bob -> Dave : $500", # T2
    "Dave -> Eve : $100", # T3
    "Eve -> Alice : $300", # T4
    "Roo -> Bob : $50", # T5
]

def build_merkle_tree(txns):
    level = [sha256_hash(t) for t in txns]
    print("Level 0 (leaf transaction hashes):")
    for i, h in enumerate(level):
        print(f" T{i+1}: {txns[i]} -> {h}")

    level_num = 1
    while len(level) > 1:
        if len(level) % 2 == 1:
            level.append(level[-1]) # duplicate last node if odd count
        next_level = []
        print(f"\nLevel {level_num} (pairwise combined hashes):")
        for i in range(0, len(level), 2):
```

Program 3 result: the Golden Nonce (53370) found for a 4-leading-zero difficulty, and the resulting hash `0000b618a680...` Program 4 (Merkle Tree) begins with the 5 sample transactions.

Jupyter Experiment_1 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab BC (venv)

```
next_level = []
print(f"\nLevel {level_num} (pairwise combined hashes):")
for i in range(0, len(level), 2):
    combined = level[i] + level[i + 1]
    parent_hash = sha256_hash(combined)
    print(f"  hash(H{i} + H{i+1}) -> {parent_hash}")
    next_level.append(parent_hash)
level = next_level
level_num += 1

return level[0]

merkle_root = build_merkle_tree(transactions)
print(f"\nMerkle Root: {merkle_root}")
```

Level 0 (leaf transaction hashes):
T1: 'Alice -> Bob : \$200' -> f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
T2: 'Bob -> Dave : \$500' -> 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cddb5b46
T3: 'Dave -> Eve : \$100' -> 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
T4: 'Eve -> Alice : \$300' -> cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
T5: 'Roo -> Bob : \$50' -> cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Level 1 (pairwise combined hashes):
hash(H0 + H1) -> 471608d3d6f2a642ff3a84dfaceaebb1c271a9aed5b37a82d61c4784558e80e9
hash(H2 + H3) -> a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
hash(H4 + H5) -> 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Level 2 (pairwise combined hashes):
hash(H0 + H1) -> ae847393c3dff60e1d95cfdabb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
hash(H2 + H3) -> 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Level 3 (pairwise combined hashes):
hash(H0 + H1) -> 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Merkle Root: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Observations

- SHA-256 produces a fixed-length 64-character hex digest regardless of input length, and even a 1-character change in input produces a completely different hash

Program 4: Level 0 leaf hashes for all 5 transactions, followed by Levels 1–3 pairwise-combined hashes as the tree is built upward.

Jupyter Experiment_1 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab BC (venv)

```
level = next_level
level_num += 1

return level[0]

merkle_root = build_merkle_tree(transactions)
print(f"\nMerkle Root: {merkle_root}")
```

Level 0 (leaf transaction hashes):
T1: 'Alice -> Bob : \$200' -> f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
T2: 'Bob -> Dave : \$500' -> 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cddb5b46
T3: 'Dave -> Eve : \$100' -> 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
T4: 'Eve -> Alice : \$300' -> cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
T5: 'Roo -> Bob : \$50' -> cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Level 1 (pairwise combined hashes):
hash(H0 + H1) -> 471608d3d6f2a642ff3a84dfaceaebb1c271a9aed5b37a82d61c4784558e80e9
hash(H2 + H3) -> a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
hash(H4 + H5) -> 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Level 2 (pairwise combined hashes):
hash(H0 + H1) -> ae847393c3dff60e1d95cfdabb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
hash(H2 + H3) -> 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Level 3 (pairwise combined hashes):
hash(H0 + H1) -> 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Merkle Root: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Observations

- SHA-256 produces a fixed-length 64-character hex digest regardless of input length, and even a 1-character change in input produces a completely different hash (avalanche effect).
- The Golden Nonce for a 4-leading-zero difficulty required a noticeable number of trial hashes, illustrating why Proof-of-Work is computationally expensive by design.
- Because the transaction count (5) was odd, the last hash was duplicated at Level 0 to pad it to an even count before pairwise combination — the standard Merkle Tree convention.
- The final Merkle Root is a single 64-character hash that uniquely fingerprints all 5 transactions; changing any one transaction would change every hash on the path.

The completed Merkle Tree: Level 3 produces the single Merkle Root `96ee90df...`, followed by the written observations.

Conclusion

This experiment implemented the four core cryptographic building blocks of a blockchain from scratch: SHA-256 hashing, nonce-based target hashing, Proof-of-Work puzzle solving, and Merkle Tree construction. Finding the Golden Nonce for a 4-leading-zero difficulty took 53,370 trial hashes, concretely demonstrating why Proof-of-Work is computationally expensive by design and why difficulty (the number of required leading zeros) directly controls how much work mining requires. Building the Merkle Tree over the 5 sample transactions — hashing each transaction, duplicating the last leaf to handle the odd count, and recursively combining pairs — produced a single Merkle Root that uniquely fingerprints the entire transaction set, illustrating how a blockchain can compactly and verifiably summarize an arbitrarily large batch of transactions in one 64-character hash.